TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

TRABAJO PRÁCTICO 2 – ADAPTER

Migración de proveedor con APIs incompatibles

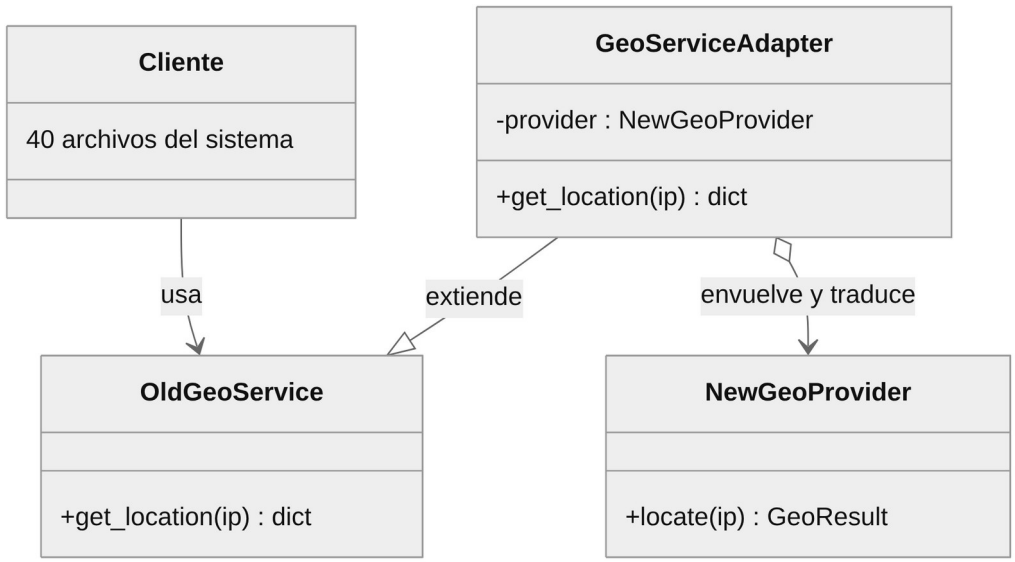
Metodología de Sistemas II – Tecnicatura Universitaria en Programación a Distancia – UTN

Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26
Fecha de entrega:	29/10/2026
Lenguaje:	Python 3 (sin librerías externas)

1. El problema

Todo el sistema (40 archivos) está escrito contra OldGeoService, que devuelve un diccionario con lat, lng, city y country. El proveedor nuevo, NewGeoProvider, tiene otro método (locate) y devuelve un objeto con otra estructura. No se pueden modificar ninguna de las dos clases ni los 40 archivos.

2. Diagrama UML



El cliente sigue dependiendo solo de OldGeoService. GeoServiceAdapter extiende OldGeoService, así que el cliente lo puede usar sin saberlo, y por dentro tiene un NewGeoProvider al que le pide los datos y los traduce.

3. Antes y después

Para simular los 40 archivos usé cliente.py, que es **idéntico** en las dos versiones. Todos los archivos obtienen el servicio desde config.py, que es el único lugar donde se crea.

Antes

antes/config.py

```
# Único lugar donde se crea el servicio de geolocalización.
from old_geo_service import OldGeoService

geo = OldGeoService()
```



Después (único cambio en archivos existentes)

despues/config.py

```
# Único lugar donde se crea el servicio de geolocalización.
from geo_service_adapter import GeoServiceAdapter

geo = GeoServiceAdapter()   # antes: geo = OldGeoService()
```

El código cliente (sin cambios)

cliente.py (antes = después)

```
# Representa a cualquiera de los 40 archivos del sistema.
# Este archivo es IDÉNTICO antes y después del Adapter.
from config import geo


def mostrar_ubicacion(ip):
    data = geo.get_location(ip)
    print(f"IP {ip}: {data['city']}, {data['country']} (lat {data['lat']}, lng {data['lng']})")


if __name__ == "__main__":
    mostrar_ubicacion("200.45.123.10")
```

El archivo nuevo: el Adapter

despues/geo_service_adapter.py

```
# ÚNICO archivo nuevo. Por fuera se ve como OldGeoService,
# por dentro usa NewGeoProvider y traduce la respuesta al formato viejo.
from old_geo_service import OldGeoService
from new_geo_provider import NewGeoProvider


class GeoServiceAdapter(OldGeoService):
    def __init__(self, provider=None):
        # Se puede pasar el proveedor por parámetro (útil para pruebas);
        # si no se pasa, usa el nuevo.
        self.provider = provider or NewGeoProvider()

    def get_location(self, ip: str) -> dict:   # misma firma que la API vieja
        resultado = self.provider.locate(ip)
        return {
            "lat": resultado.coordinates.latitude,
            "lng": resultado.coordinates.longitude,
            "city": resultado.address.locality,
            "country": resultado.address.nation,
        }


# ¿Qué cambia si mañana llega un tercer proveedor?
# El sistema no se toca: los 40 archivos siguen llamando a get_location(ip).
# Solo cambia la traducción. Hay dos opciones:
#   1) Modificar este adapter para que llame al nuevo proveedor y traduzca su respuesta.
#   2) Si conviven varios proveedores, hacer un adapter por cada uno
#      (por ejemplo FutureGeoAdapter) y cambiar una línea en config.py.
# En los dos casos el cambio queda aislado en la capa de adaptación.
```

Clases existentes (no se modificaron, se simulan para poder probar)

old_geo_service.py

```
# API vieja: es la que usa todo el sistema. NO se puede modificar.
# (Simulada: en el sistema real llamaba a la librería sin mantenimiento.)


class OldGeoService:
    def get_location(self, ip: str) -> dict:
        return {"lat": -34.60, "lng": -58.38, "city": "Buenos Aires", "country": "Argentina"}
```

new_geo_provider.py

```
# Proveedor nuevo (de terceros). NO se puede modificar.
# Devuelve un objeto con una estructura distinta a la de la API vieja.


class Coordinates:
    def __init__(self, latitude, longitude):
        self.latitude = latitude
        self.longitude = longitude


class Address:
    def __init__(self, locality, nation):
        self.locality = locality
        self.nation = nation


class GeoResult:
    def __init__(self, coordinates, address):
        self.coordinates = coordinates
        self.address = address


class NewGeoProvider:
    def locate(self, ip: str) -> GeoResult:
        return GeoResult(Coordinates(-34.6037, -58.3816), Address("Buenos Aires", "Argentina"))
```

4. Ejecución

```
$ python antes/cliente.py
IP 200.45.123.10: Buenos Aires, Argentina (lat -34.6, lng -58.38)

$ python despues/cliente.py
IP 200.45.123.10: Buenos Aires, Argentina (lat -34.6037, lng -58.3816)
```

El mismo cliente.py funciona con los dos proveedores. Lo único que cambia son las coordenadas, que el proveedor nuevo da con más decimales, y eso muestra que los datos ahora vienen de NewGeoProvider.

5. Justificación

Elegí Adapter porque el problema es que hay dos interfaces que no encajan y ninguna se puede tocar. Reescribir los 40 archivos para usar la API nueva es mucho trabajo y riesgo de romper algo. Además, si mañana se cambia otra vez de proveedor, habría que volver a hacerlo. Una Facade no sirve, porque su idea es simplificar un subsistema complejo con una interfaz nueva, y acá lo que necesito es lo contrario: mantener exactamente la interfaz vieja. Con el Adapter el sistema sigue llamando a get_location(ip) igual que siempre, y toda la traducción queda en una sola clase. El único cambio en el código existente es una línea en config.py.

6. ¿Qué cambia si llega un tercer proveedor?

Los 40 archivos no se tocan. Si el tercer proveedor reemplaza al actual, solo hay que modificar el Adapter para que llame al nuevo y traduzca su respuesta. Si tienen que convivir varios proveedores, conviene hacer un adapter por cada uno (por ejemplo FutureGeoAdapter) y elegir cuál usar en config.py. En los dos casos el cambio queda aislado.

Sobre la implementación

Usé la variante de Adaptador de Objetos (composición), igual que el ejemplo de la teoría: el adapter hereda de OldGeoService para que el cliente lo acepte y guarda un NewGeoProvider en un atributo. Le agregué la posibilidad de recibir el proveedor por parámetro en el constructor, para poder probarlo con otro proveedor sin cambiar el código.